



Gymnázium J. K. Tyla

Úvod do agentického AI a design autonomní pipeline pro softwarový vývoj

Návrh a realizace systému DarkFactory

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

KONCEPT

Anotace

Práce se zabývá návrhem a realizací autonomního systému pro vývoj softwaru, který přebírá rutinní kroky vývojového procesu — od přijetí požadavku přes jeho interpretaci a naplánování až po vytvoření a ověření změny. Teoretická část shrnuje principy kontinuální integrace, řízení verzí a orchestrace jazykových modelů. V praktické části je popsán systém DarkFactory, jeho architektura a governance, a jsou vyhodnoceny výsledky jeho nasazení na reálných repozitářích.

Klíčová slova

autonomní systémy, softwarové inženýrství, kontinuální integrace, orchestrace agentů, jazykové modely

Annotation

This thesis deals with the design and implementation of an autonomous software engineering system that takes over the routine steps of the development process — from intake of a request through its interpretation and planning to the creation and verification of a change. The theoretical part summarises the principles of continuous integration, version control and the orchestration of language models. The practical part describes the DarkFactory system, its architecture and governance, and evaluates the results of its deployment on real repositories.

Keywords

autonomous systems, software engineering, continuous integration, agent orchestration, language models

Obsah

Prohlášení	2
Anotace	3
Klíčová slova	3
Annotation	3
Keywords	3
1 Úvod	7
1.1 Motivace	7
1.2 Cíl práce	7
1.3 Metodika	8
1.4 Struktura práce	8
2 Teoretická část	9
2.1 Řízení verzí	9
2.1.1 Větvě a jejich role	9
2.1.2 Model pull requestu	9
2.2 Kontinuální integrace	9
2.2.1 Požadované kontroly	9
2.2.2 Sestavení a artefakty	10
2.3 Jazykové modely a jejich orchestrace	10
2.3.1 Transformer a LLM	10
2.3.2 Tokeny, tokenizér a embedding	10
2.3.3 Multimodální modely	11
2.3.4 Context	12
2.3.5 Prompt a kontextové inženýrství	13
2.3.6 Agent vs Chatbot	14
2.3.7 Harness a System Prompt	14
2.3.8 ReAct smyčka	14
2.3.9 Vyvolávání nástrojů	15
2.3.10 Dovednosti, skripty a zachytané body	16
2.3.11 MCP Server	17
2.3.12 Meta Harness	17
2.3.13 Subagenti	18
2.3.14 Workflows neboli Graph Engineering	18
2.4 Human in the loop neboli člověk ve smyčce	19
2.4.1 Schvalovací body	19

2.4.2	Dohledatelnost	19
2.4.3	Hlášení selhání	19
3	Praktická část	20
3.1	Cíl a rozsah systému	20
3.2	Architektura	20
3.2.1	Sdílení místo kopírování	21
3.2.2	Jediný konfigurační soubor	21
3.3	Životní cyklus požadavku	22
3.4	Popis prostředí repozitáře	24
3.4.1	Domény a prostředí	24
3.4.2	Deklarace jako doplněk rozpoznávání	24
3.5	Orchestrace napříč poskytovateli	25
3.6	Ověřování změn	26
3.7	Hlášení selhání	26
3.8	Automaticky generovaná dokumentace	26
3.9	Systém revizních značek pro lidský dohled nad textem	27
4	Výsledky a diskuse	29
4.1	Metodika ověření	29
4.2	Sjednocení pracovních postupů	29
4.3	Rozšíření na texty	30
4.4	Chyby, které se projeví až v provozu	30
4.5	Diskuse	32
4.6	Omezení	32
5	Závěr	33
	Seznam příloh	35
A	Obsah přiloženého média	36
B	Referenční specifikace konfiguračního souboru	37
C	Protokol revizních značek v sazebním systému Typst	40
C.1	Vizuální reprezentace jednotlivých prvků v sazbě	40
	Seznam zdrojů	42

Seznam obrázků a tabulek

Obrázek 1	Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.	15
Obrázek 2	Dekompozice komponent systému DarkFactory: třívrstvá architektura propojující platformu GitHub, výkonné skriptové jádro a izolované běhové prostředí s modely.	20
Obrázek 3	Stavový diagram životního cyklu požadavku v systému DarkFactory: přechody mezi stavy od založení issue přes schvalovací brány (Human Gates), tvorbu větve a PR až po automatické sloučení a uzavření úkolu.	23
Tabulka 1	Rozpoznávání prostředí podle souboru popisujícího balíček.	24
Tabulka 2	Počet vlastních skriptů a rozsah odstraněného kódu před sjednocením a po něm.	29

1 Úvod

Vývoj softwaru se za posledních dvacet let z velké části zautomatizoval. Sestavení programu, spuštění testů, kontrola stylu i nasazení do provozu dnes obstarávají stroje a nikdo je nepovažuje za práci hodnou lidského času. Jeden krok však zůstal ruční: samotná změna zdrojového kódu. Právě tam se tvoří většina prodlev — mezi okamžikem, kdy někdo popíše požadavek, a okamžikem, kdy se změna dostane k uživatelům.

Výrobní průmysl zná pojem *dark factory*, tedy „temná továrna“: provoz, který běží bez lidské obsluhy, a proto v něm nemusí svítit. Nejde o představu úplného vyloučení člověka — i temná továrna má konstruktéry, kteří rozhodují, co se bude vyrábět — nýbrž o vyloučení člověka z opakujících se úkonů. Tato práce zkoumá, nakolik lze týž princip uplatnit ve vývoji softwaru a kde jsou jeho hranice.

1.1 Motivace

S rozšířením velkých jazykových modelů se objevila možnost automatizovat i psaní kódu. Většina dostupných nástrojů však řeší jen dílčí krok: vygenerují návrh změny, který někdo musí zasadit do procesu, ověřit a schválit. Chybí popis toho, jak má takový nástroj zapadnout do vývojového procesu jako celku — kdo schvaluje, co se stane při selhání a jak se pozná, že je výsledek správný.

Právě tato mezera je předmětem práce. Zajímavá není otázka, zda model dokáže napsat kód; to je dnes doloženo. Zajímavá je otázka, jaké okolí musí kolem takového modelu vzniknout, aby jeho výstupu bylo možné důvěřovat.

1.2 Cíl práce

Cílem této práce je navrhnout, realizovat a ověřit systém, který automatizuje vývojový proces od přijetí požadavku po vytvoření ověřené změny, aniž by se vzdal lidského schválení v rozhodujících bodech.

Dílčí cíle:

1. Popsat současný stav automatizace vývoje softwaru a orchestrace jazykových modelů.
2. Navrhnout architekturu systému, který provede požadavek celým procesem.
3. Systém realizovat a nasadit na reálné repozitáře.
4. Vyhodnotit jeho chování a pojmenovat omezení, na která v provozu narazil.



Návrh na vylepšení: Formulace výzkumných otázek: Pro zvýšení vědeckého a metodického přínosu práce u obhajoby doporučuji pod cíle práce doplnit 2–3 explicitní výzkumné otázky (např. zda lze dosáhnout bezobslužného běhu pipeline při zachování striktních deterministických záruk a jaké jsou limitující faktory současných LLM při samostatné práci nad rozsáhlým repozitářem).

1.3 Metodika

Práce je z povahy tématu konstrukční: hlavním výstupem je funkční systém, nikoli měření. Postup odpovídá vývoji softwaru — po nastudování východisek následoval návrh, realizace a nasazení na reálné repozitáře, přičemž zjištění z provozu se vracela zpět do návrhu. Ověření proto neprobíhalo formou experimentu s kontrolní skupinou, nýbrž sledováním chování systému v provozu. Sledovány byly zejména nalezené chyby, neboť právě ty ukazují na rozdíl mezi předpokladem a skutečností.

 **Hlubková kritika / Oponentura:** Metodologická zranitelnost u obhajoby: Tvrzení, že „hlavním výstupem je funkční systém, nikoli měření“, je z hlediska exaktního výzkumu v softwarovém inženýrství nepřijatelná berlička. Pouhé sledování chování systému bez kontrolní skupiny či standardizovaných srovnávacích benchmarků (např. SWE-bench, `pass@k` či měření poměru úspěšnosti) činí celou práci zranitelnou vůči námitce, že jde pouze o subjektivní případovou studii autora nad vlastními třemi repozitáři. Oponent se zeptá: Jaká je statistická úspěšnost vyřešení issue na první pokus? Kolik tokenů pipeline spálí na banální chybu? Kde je baseline srovnání s manuálním vývojem?

 **Návrh na vylepšení:** Doporučuji v metodice explicitně uvést zkoumané repozitáře (vlastní systém DarkFactory, vícejazyčná aplikace omnis a menší projekt ChessWithQuests) a specifikovat, že ověření probíhá sledováním reálných běhů automatizovaného GitHub Actions workflow nad reálnými issues a pull requesty.

1.4 Struktura práce

Kapitola 2 shrnuje teoretická východiska: řízení verzí, kontinuální integraci, architekturu a orchestraci jazykových modelů a princip zapojení člověka do smyčky (*Human-in-the-loop*). Kapitola 3 popisuje vlastní systém DarkFactory — jeho architekturu, životní cyklus požadavku a způsob, jímž rozpoznává obsah repozitáře. Kapitola 4 hodnotí výsledky nasazení včetně chyb, které se projeví až v provozu, a kapitola 5 je shrnuje.

2 Teoretická část

Tato kapitola vymezuje pojmy, o které se opírá praktická část: řízení verzí, kontinuální integraci, orchestraci jazykových modelů a princip zapojení člověka do smyčky (*Human-in-the-loop*). Cílem není vyčerpávající přehled, nýbrž zavedení pojmů v podobě, v jaké s nimi pracuje navržený systém.

2.1 Řízení verzí

Systém pro řízení verzí uchovává historii změn zdrojového kódu. Distribuovaný model, jehož nejrozšířenějším zástupcem je Git, se od centralizovaného liší tím, že každý vývojář má úplnou kopii historie (1). Změny lze proto vytvářet a zkoumat i bez spojení se serverem a slučovat je až ve chvíli, kdy jsou hotové.

2.1.1 Větvě a jejich role

Větev je pojmenovaný ukazatel na určitý stav historie. Práce na nové vlastnosti probíhá ve větví oddělené od hlavní vývojové linie, takže rozpracovaný stav neovlivní ostatní. Tento postup má i důsledek pro automatizaci: dokud změna existuje pouze ve větví, lze ji libovolně ověřovat, aniž by hrozila škoda.

2.1.2 Model pull requestu

Sloučení větve do hlavní linie se ve většině dnešních projektů odehrává prostřednictvím *pull requestu* — návrhu změny, který lze komentovat, ověřovat a schvalovat. Pull request je proto přirozeným místem, kde se uplatňuje kontrola kvality, a zároveň místem, kam lze vložit schvalovací bod pro člověka.

2.2 Kontinuální integrace

Kontinuální integrace (angl. *continuous integration*) je praxe, při níž se každá změna automaticky sestaví a otestuje (2). Namísto dlouhých období, kdy se změny hromadí a slučují až na konci, se ověřuje průběžně a v malých dávkách, takže chyba je odhalena blízko svému vzniku.

2.2.1 Požadované kontroly

Ke kontinuální integraci patří pojem *požadovaných kontrol* (angl. *required checks*): množina úloh, které musí skončit úspěšně, jinak nelze změnu sloučit. Tím se z kvality stává vlastnost vynucovaná strojem, nikoli pouze dohodou mezi vývojáři.

Návrh požadovaných kontrol má jedno nezřejmé úskalí. Úloha, která se za jistých okolností „přeskočí“, nehlásí žádný výsledek; je-li přitom uvedena mezi požadovanými, zablokuje slučování napořád. Kontrola proto musí vždy skončit nějakým závěrem — i kdyby jím bylo konstatování, že v daném repozitáři není co dělat.

2.2.2 Sestavení a artefakty

Výsledkem sestavení bývá *artefakt*: spustitelný soubor, knihovna, nebo — jak ukazuje praktická část — vysázený dokument. Automatizace vydávání verzí spojuje artefakt se značkou v historii, takže ke každé vydané verzi existuje doložitelný výstup.

2.3 Jazykové modely a jejich orchestrace

2.3.1 Transformer a LLM

Transformer je architektura hlubokých neuronových sítí představená společností Google v roce 2017 v rámci výzkumu strojového překladu. Na rozdíl od starších architektur (např. rekurentních sítí RNN), které sekvence zpracovávaly krok po kroku a trpěly ztrátou dlouhodobého kontextu, využívá transformer mechanismus zvaný *attention* (pozornost). Ten modelu umožňuje paralelně vyhodnocovat sémantické souvislosti mezi všemi tokeny v sekvenci bez ohledu na jejich vzájemnou vzdálenost.

LLM neboli *Large Language Model* (velký jazykový model) je rozsáhlá neuronová síť postavená na architektuře transformera a předtrénovaná na textových datech o objemu stovek miliard až bilionů tokenů. Díky mechanismu pozornosti model dokáže zachytit hluboké syntaktické i sémantické struktury přirozeného jazyka, což se navenek projevuje schopností generalizace, abstrakce a generování korektního zdrojového kódu.

Tento architektonický přelom popsala publikace *Attention Is All You Need* (3). Původní model byl koncipován jako **Encoder-Decoder** (kodér-dekodér) pro strojový překlad: obousměrný kodér nejprve zkomprimoval celou vstupní větu a dekodér za pomoci křížové pozornosti (*cross-attention*) generoval překlad v cílovém jazyce.

Ve vývoji softwaru a moderních agentních systémech se však naprostým standardem stala architektura **Decoder-only** (např. GPT, Claude, LLaMA či DeepSeek). Tyto modely pracují čistě autoregresivně — predikují vždy následující nejpravděpodobnější token na základě celého předcházejícího kontextu. Instrukce, pravidla, kontext repozitáře i rozepsaný kód tvoří jedinou společnou sekvenci, což umožňuje plynulé doplňování kódu i přímé generování volání nástrojů. Architektura pouze s dekodérem navíc vykazuje vynikající vlastnosti při škálování parametrů a efektivní správě KV cache v dlouhých kontextech.

2.3.2 Tokeny, tokenizér a embedding

Text, se kterým člověk pracuje ve formě slov a vět, není pro neuronovou síť přímo srozumitelný. Model interně operuje pouze s čísly a maticovými operacemi. Aby bylo možné přirozený jazyk zpracovat, musí projít procesem tokenizace a následného převodu na vektorové reprezentace.

Základní jednotkou, kterou model vnímá, je **token**. Token nemusí odpovídat celému slovu ani jednotlivému znaku — moderní jazykové modely využívají tzv. sub-word tokenizaci (nejčastěji algoritmy jako Byte-Pair Encoding či WordPiece). Běžná slova jsou často repre-

zentována jediným tokenem, zatímco méně častá slova, odborné výrazy nebo slova v jazycích s bohatou morfologií a diakritikou (jako je čeština) jsou rozložena do více dílčích tokenů. Průměrně jeden token v angličtině odpovídá přibližně 3 až 4 znakům. V češtině je spotřeba tokenů na slovo znatelně vyšší, což má přímý dopad na efektivní kapacitu kontextového okna i výpočetní náklady inference.

Převod mezi surovým textovým řetězcem a posloupností celočíselných identifikátorů (token IDs) zajišťuje **tokenizér**. Jedná se o deterministický program, který vychází z pevně daného slovníku (tzv. vocabulary, obvykle čítajícího 32 000 až 128 000 unikátních tokenů). Tokenizér provádí obousměrný proces: při vstupu rozseká text na tokeny a přiřadí jim číselné indexy, při výstupu naopak generované indexy skládá zpět do souvislého textu (tzv. detokenizace).

Samotné číslo tokenu však nenese žádnou informaci o jeho významu. Proto následuje vrstva zvaná **embedding** (vektorové vnoření). Každý token je převeden na spojitý vícerozměrný vektor (typicky o dimenzi 4 096 či 8 192 čísel). V tomto geometrickém prostoru jsou slova s podobným významem či kontextem umístěna blízko u sebe (např. měreno kosinovou podobností).

Jednotlivé geometrické směry a posuny v prostoru navíc odpovídají konkrétním sémantickým relacím a abstraktním vlastnostem. Zjednodušeným a klasickým příkladem fungování tohoto prostoru je vektorová aritmetika pojmů: pokud vezmeme vektor reprezentující pojem „žena“ a přičteme k němu vektor reprezentující posun k „panovnickému stavu / královskému statusu“, výsledný bod v prostoru leží nejbližší reprezentaci slova „královna“ ($\vec{v}(\text{žena}) + \vec{v}(\text{královský status}) \approx \vec{v}(\text{královna})$). Model tak nepracuje se slovy jako s izolovanými symboly, ale manipuluje se vztahy mezi nimi jako s algebraickými posuny ve vícerozměrném prostoru — což se v programování projevuje schopností zachytit vztahy mezi deklarací proměnné, jejím použitím a typovým kontextem.

Aby architektura transformeru zohlednila také pořadí tokenů v sekvenci, přičítá se k sémantickému embeddingu poziční kódování (positional encoding, dnes standardně rotační embedding RoPE). Teprve takto vzniklé vektory vstupují do mechanismu pozornosti k dalšímu výpočtu.

 **Návrh na vylepšení:** Vhodné doplnit malou srovnávací tabulku či praktický příklad: kolik tokenů spotřebuje identický větný význam v češtině oproti angličtině (např. pomocí knihovny tiktoken), což krásně podloží argumentaci o efektivním využití kontextového okna a nákladech na inference.

2.3.3 Multimodální modely

Architekturu transformeru lze aplikovat i mimo oblast zpracování přirozeného jazyka. V moderní praxi lze tokenizovat v podstatě jakýkoli diskrétní či spojitý signál — ať už jde o

rastrový obraz, sekvenci snímků videa, zvukové vlny nebo trajektorie pohybů v robotice. Zásadní posun nastává ve chvíli, kdy model disponuje oddělenými projekčními vrstvami pro různé typy vstupů (např. kombinací textového tokenizéru a vizuálního enkodéru, jako je ViT — *Vision Transformer*) a mechanismus pozornosti (*cross-attention*) operuje společně nad tokeny textu i obrazu. Model tak dokáže propojovat vizuální a textové sémantické reprezentace ve sdíleném vektorovém prostoru.

V kontextu automatizovaného vývoje softwaru a autonomních pipeline (jako je systém DarkFactory) se multimodální schopnosti uplatňují ve třech klíčových oblastech:

1. **Vizuální regresní testování:** Model dokáže porovnat referenční snímek uživatelského rozhraní se stavem vygenerovaným v CI (např. při testování webových komponent bezhlavým prohlížečem) a identifikovat nežádoucí posuny rozvržení či stylové chyby.
2. **Diagnostika chyb z artefaktů CI:** Při selhání integračních testů může pipeline předat agentovi screenshot chybové obrazovky nebo interaktivního prvku, z něhož agent rozpozná příčinu selhání snáze než z pouhého textového stack trace.
3. **Interpretace grafických zadání:** Vývojář může v zadání úkolu (GitHub Issue) specifikovat požadovanou změnu formou náčrtku, wireframu či diagramu komponent. Multimodální agent takový grafický podklad přímo zanalyzuje a převede jej na strukturovaný kód.

2.3.4 Context

2.3.4.1 Turn

Každému kroku mezi modelem a uživatelem se říká turn, nebo specificky model turn pro každé spuštění inference.

2.3.4.2 Context Window

Model je schopen přijmout pouze omezený objem vstupu; tomuto limitu se říká kontextové okno (*Context Window*) a vyjadřuje se v počtu tokenů. Na rozdíl od slovníku (*vocabulary*), který vymezuje pouze repertoár známých tokenů, je maximální délka kontextového okna určena architekturou pozičního kódování (např. škálováním frekvenčních bází v RoPE) a hardwarovou náročností mechanismu pozornosti — kvadratickou složitostí $O(N^2)$ a velikostí alokované KV cache v operační či grafické paměti.

2.3.4.3 Compaction

Představuje proces sumarizace a zkrácení historie, který řídicí harness iniciuje ve chvíli, kdy zaplnění kontextového okna dosáhne stanoveného prahu. Zpravidla jde o vyvolání modelu se specifickou systémovou instrukcí pro bezetrátovou syntézu dosavadního průběhu sezení a kompletním protokolem dosavadní komunikace. Výsledný zkrácený kontext následně v kontextovém okně nahradí původní rozsáhlou historii kroků.

2.3.4.4 Context Rot

 **Hlubková kritika / Oponentura:** Teoretická naivita rekurzivní komprese: Popsaný mechanismus komprese (Compaction) se v textu tváří jako elegantní a bezproblémové řešení, v reálu jde však o destruktivní ztrátovou kompresi. Model při rekurzivním zkracování historie trpí silným konfirmačním zkreslením — sumarizuje to, co sám považuje za podstatné, čímž nevratně maže přesná čísla řádků, jemné sémantické hrany zadání, negativní mantinely („tohle nikdy neměň“) a detaily chybových hlášení. Práce se vůbec nezabývá fundamentálním fenoménem sémantického posunu (*semantic drift*) po několika kolech komprese, ani moderními bezztrátovými alternativami (hierarchická RAG paměť, persistentní graf stavu projektu či selective KV cache eviction).

~~Je koncept, který byl pozorován v praxi, kdy modely ztrácí inteligenci a nejsou schopni si zapamatovat detaily ze začátku čím plnější je CW.~~ Označuje empiricky zdokumentovanou degradaci schopnosti modelu věnovat rovnoměrnou pozornost všem částem historie (tzv. jev *Lost in the Middle* (4)). Čím plnější je kontextové okno, tím méně jsou modely schopny spolehlivě vybavovat jemné detaily z úvodu sezení a dodržovat negativní omezující podmínky zadání.

2.3.4.5 KV Caching

KV caching slouží ke snížení výpočetní náročnosti modelu při generování tokenů. Místo toho, abychom při každém kroku inference znovu od začátku počítali pozornost pro celou dosavadní konverzaci, ponechává inferenční engine v paměti uložené již vypočtené vektory klíčů a hodnot (*Key-Value pairs*) a v každém kroku počítá a přidává pouze nově vygenerovaný token.

2.3.5 Prompt a kontextové inženýrství

Prompt engineering (inženýrství promptů) je disciplína zaměřená na systematický návrh, formulaci a optimalizaci textových instrukcí předkládaných modelu (5). V autonomních agentních systémech nepředstavuje prompt pouhou volnou konverzaci, ale slouží jako závazný kontrakt vymezující chování, práva a bezpečnostní mantinely agenta. Mezi klíčové techniky patří:

- **Systémový prompt (System Prompt):** Základní direktiva definující identitu agenta, dostupné nástroje a striktní provozní pravidla (např. pravidla pro zachování neměnnosti existujících testů, konvence pro formát commitů či zákaz destruktivních operací v repozitáři).
- **Few-shot prompting:** Technika vložení několika vzorových příkladů (vstup-výstup) přímo do kontextu. Tím model získá konkrétní představu o požadovaném schématu (např. syntaxi konfiguračního souboru `darkfactory.json`) bez nutnosti dodatečného dotrénování parametrů.

- **Chain-of-Thought (myšlenkový řetězec):** Vedení modelu k explicitní formulaci mezikroků a vnitřní dedukce před vygenerováním konečného kódu či akce. Rozklad komplexního zadání na postupné logické kroky zásadně potlačuje halucinace a tvoří základ fáze rozvahy (*Thought*) v cyklu ReAct.
- **Injekce dynamického kontextu:** Průběžné doplňování promptu o aktuální stav repozitáře, stromovou strukturu souborů, chybové výpisy kompilátoru a výsledky testů, díky čemuž agent operuje nad reálnými fakty namísto odhadů.

2.3.6 Agent vs Chatbot

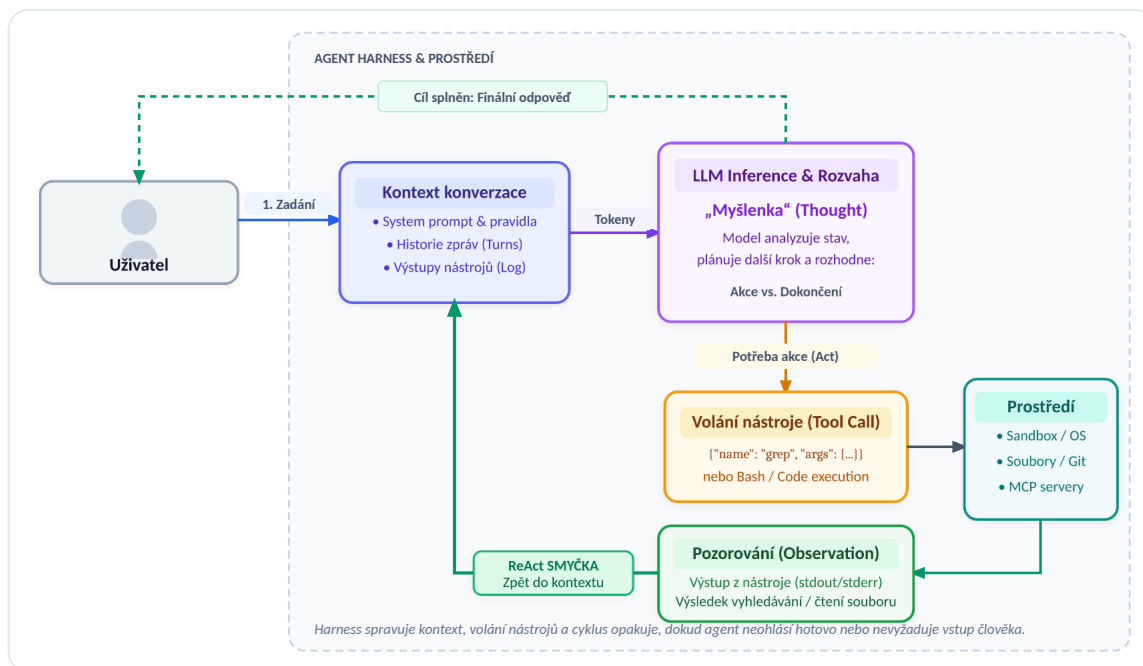
Rozdíl mezi chatbotem a agentem je v principu, že chatbot je schopný pouze textu v konverzaci AKA chatu, agent se z něj stává ve chvíli kdy je schopný dělat více než to, tedy když mu přidáme nástroje, se kterými může například prohlédávat web, upravovat soubory nebo i přímo ovládat váš počítač. Fundamentální rozdíl mezi chatbotem a agentem spočívá v rozsahu interakce s okolním světem: zatímco chatbot operuje výhradně v uzavřeném konverzačním rozhraní a generuje textové odpovědi, agentem se systém stává ve chvíli, kdy je vybaven sadou výkonných nástrojů (*tools*). Prostřednictvím nich dokáže aktivně zkoumat repozitář, vyhledávat informace na webu, modifikovat soubory a autonomně vykonávat příkazy v cílovém výpočetním prostředí.

2.3.7 Harness a System Prompt

Harness je řídicí program obklopující jazykový model. Odlišuje se od inferenčního jádra (*inference engine*), které provádí samotné maticové výpočty sítě: harness má na starost rozhraní mezi uživatelem a agentem či chatbotem, předávání systémového promptu, bezpečné spouštění nástrojů a řízení iterativní smyčky neboli ReAct smyčky. Mezi typické příklady patří webová aplikace ChatGPT, terminálové rozhraní Claude Code, desktopová aplikace Codex a další agentní prostředí.

2.3.8 ReAct smyčka

Smyčka ReAct (*Reasoning and Acting*) tvoří základní stavební kámen, který z jazykového modelu vytváří autonomního agenta namísto pouhého pasivního generátoru textu (6). Princip nespočívá v pouhém přečtení vstupu a vykonání akce, nýbrž v soustavné alternaci vnitřní rozvahy (*Reasoning* neboli *Thought*) a vnějšího jednání (*Acting* neboli *Tool Call*). Model v každém kroku nejprve formuluje hypotézu či myšlenkový postup, na jehož základě provede cílenou akci. Po vykonání akce harness připojí vrácený výsledek (*Observation*) do kontextového okna na konec logu konverzace. V navazujícím kroku inference model zhodnotí nový stav a cyklus opakuje, dokud úkol není vyřešen nebo dokud nerozhodne, že má dostatek podkladů pro předání finální odpovědi uživateli. Tímto způsobem je dosaženo adaptivního řešení problémů namísto jednorázové slepé generace.



Obrázek 1: Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.

Diagram na Obrázek 1 znázorňuje základní iterativní cyklus moderních autonomních agentů. Po přijetí uživatelského zadání dochází v rámci inference k fázi rozvahy (*Reasoning*), kdy model formuluje vnitřní myšlenkový postup (*Thought*). Pokud je k vyřešení kroku zapotřebí externí akce, model vygeneruje strukturované volání nástroje (*Tool Call*). Řídící harness tento požadavek zachytí, bezpečně vykoná v cílovém prostředí (terminál, souborový systém, API či MCP server) a vrácený výsledek (*Observation*) připojí na konec kontextového okna. Celý aktualizovaný kontext je následně předložen modelu v další iteraci, dokud není úkol kompletní a výsledek předán uživateli. Tento princip byl formálně zaveden v práci *ReAct: Synergizing Reasoning and Acting in Language Models* ((6), [arXiv:2210.03629](https://arxiv.org/abs/2210.03629)).

2.3.9 Vyvolávání nástrojů

Abychom z jazykového modelu vytvořili autonomního agenta, musíme jej vybavit rozhraním pro interakci s okolním prostředím — tzv. nástroji (*tools*). V praxi existují dva převažující přístupy k realizaci nástrojů:

1. **Strukturované volání nástrojů (Function / Tool Calling v JSON):** Model ve svém výstupu zanechá strukturované volání odpovídající schématu zadanému v definici nástroje. Řídící harness toto volání zachytí, vykoná požadovanou funkci a vrátí výsledek zpět agentovi opět jako strukturovaná data. Tato možnost je spolehlivá pro jednoúčelové operace (např. integraci do podnikových systémů či zákaznické podpory — angl. *customer support*). Pro komplexní softwarový vývoj však představuje nevýhodu vysoká režie formátu JSON: velké množství formátovacích znaků se převádí na tokeny, což zbytečně plní kontextové okno a může vést k degradaci pozornosti (*Context Rot*).

2. **Přímé spouštění kódu (Code Execution):** Druhou možností je poskytnout agentovi plnohodnotné běhové prostředí (např. Bash, Python či TypeScript). Agent vygeneruje kód pro splnění svého záměru a harness jej spustí buď po jednotlivých příkazech, nebo jako ucelený skript. Z bezpečnostních důvodů je nezbytné, aby agent nepracoval přímo na nechráněném hostitelském počítači, nýbrž ve virtuálním prostředí odděleném bezpečnostní vrstvou — tzv. sandboxu (pískovišti). Tento způsob je pro softwarový vývoj nejefektivnější.

 **Hlubková kritika / Oponentura:** Iluzorní bezpečnost pískoviště: Druhý přístup (přímé spouštění kódu v kontejneru) práce nekriticky prezentuje jako „nejefektivnější“, zamlčuje však gigantická bezpečnostní a operační rizika. Běžný Docker kontejner není plnohodnotná bezpečnostní hranice (*security boundary*). Spuštění netestovaného kódu generovaného modelem s přístupem k síti otevírá prostor pro útoky typu Server-Side Request Forgery (SSRF), úniky tajných environmentálních proměnných (GitHub tokeny, API klíče k LLM poskytovatelům) přes postranní síťové kanály a nevratné poškození repozitáře. Práce postrádá jakoukoli analýzu formální izolace (např. microVM architektury jako AWS Firecracker, gVisor či striktní network namespaces) a nezodpovídá otázku, co zabráni kompromitovanému modelu zneužít CI runner jako odrazový můstek pro kompromitaci infrastruktury.

2.3.10 Dovednosti, skripty a záchytné body

Tyto standardy vznikly proto, aby vývojáři mohli modulárně upravovat chování agenta a rozšiřovat jeho schopnosti pro specifické doménové úlohy:

- **Dovednost (Skill):** Samostatný adresář sdružující instrukce, reference a pomocné soubory. Klíčovým prvkem je soubor `SKILL.md`, který využívá hlavičku v metadatovém formátu YAML frontmatter (ohraňovanou trojicí pomlček `---`). V hlavičce je definován název a stručný popis dovednosti. Řídící harness do základního systémového promptu vkládá pouze tato metadata; samotný text podrobného návodu se do kontextu načte až ve chvíli, kdy agent danou dovednost vyvolá. Tím se efektivně šetří kapacita kontextového okna.
- **Skript (Script):** Jednouúčelový spustitelný program (např. v jazyce Python či Bash). Skripty jsou pro efektivitu práce vitální: agent nemusí generovat každý příkaz z paměti, ale spouští deterministické a otestované postupy pro analýzu a transformaci kódu.
- **Záchytný bod (Hook):** Skript, který se v harnessu automaticky vyvolá při určité systémové události či stavovém přechodu (např. při inicializaci sezení, před odesláním promptu modelu nebo při selhání nástroje). Hooky umožňují vynucovat bezpečnostní pravidla a logování nezávisle na vůli samotného modelu.

2.3.11 MCP Server

~~Model Context Protocol vznikl v laboratoři Anthropicu jako způsob pro efektivní interakci agentů s API servery. Ve své podstatě se jedná o nástroje postavené na základě externího API tedy to sjednotí různé standardy do jednoho a přidá „context“ efektivně metadata pro každou přítomnou metodu, aby agent věděl co má od nich čekat a nemusel sám přijít na to jak backend funguje.~~

Model Context Protocol vznikl v laboratořích společnosti Anthropic jako standardizovaný způsob pro efektivní interakci agentů s API servery. Ve své podstatě se jedná o nástroje postavené na rozhraní externího API, tudíž protokol sjednocuje různé standardy do jediného otevřeného rozhraní a přidává kontext — metadata pro každou přítomnou metodu, aby agent předem věděl, co od ní očekávat, a nemusel sám odhadovat fungování backendu.

Praktický význam specifikace Model Context Protocol (7) spočívá ve sjednocení dříve fragmentovaného ekosystému proprietárních rozhraní a jednoúčelových integračních skriptů. Řídící harness se díky standardu stává univerzálním klientem a veškeré externí schopnosti, datové zdroje či nástroje se integrují jako samostatné procesy — tzv. MCP servery. Komunikace probíhá přes standardizovaný protokol JSON-RPC, a to buď lokálně prostřednictvím standardního vstupu a výstupu (`stdio`), nebo vzdáleně pomocí protokolu HTTP se Server-Sent Events (`SSE`). Tento modulární přístup striktně odděluje běhové prostředí agenta od samotné implementace nástrojů: libovolnou schopnost (např. přístup k databázi, vyhledávání v repozitáři nebo správu GitHub úkolů) stačí naimplementovat jednou a lze ji okamžitě zpřístupnit jakémukoli kompatibilnímu agentovi bez nutnosti zásahu do kódu samotného harnessu.

2.3.12 Meta Harness

~~V reálném světě se ukázalo že nejefektivnější harness je takový, který dá samotnému agentovi rozhodnou moc nad jeho architekturou a možnost kontinální změny tedy evoluce. Toto je sice velice efektivní pro maximalizování práce agenta, ale v reálném světě se toto dá nasadit pouze pro osobního agenta. Například pro customer support botu firma určitě nestojí o to aby jejich model přišel na to jak dělat něco jiného.~~

V reálném světě se ukázalo, že nejefektivnější harness je takový, který dá samotnému agentovi rozhodovací pravomoc nad vlastní architekturou a umožňuje kontinuální změnu, tedy samořízenou evoluci (8). Tento přístup je mimořádně efektivní pro maximalizování pracovní kapacity agenta, v praxi se však hodí především pro osobního agenta. U komerčních nasazení, jako je zákaznická podpora (*customer support*), provozovatel naopak striktně vyžaduje deterministické mantinely a vylučuje, aby model modifikoval své vlastní provozní instrukce.

2.3.13 Subagenti

 **Strukturální upozornění:** Strukturální torzo: Sekce o subagentech čítá jedinou větu, ačkoli jde o klíčový koncept moderní víceagentní orchestrace. Je nezbytné buď podkapitolu strukturálně rozpracovat (popsat komunikační vzory, hierarchii rolí, asynchronní zasílání zpráv a delegování specializovaných úloh), nebo ji sloučit s bezprostředně navazující sekci „Workflows neboli Graph Engineering“.

Když dáme agentovi nástroj s možností vyvolat jiného agenta, zadat mu úlohu, interagovat s ním a sledovat jeho progress drasticky zvýšíme jeho efektivnost pro rozsáhlé úlohy. Tomuto se říká „subagents“.

2.3.14 Workflows neboli Graph Engineering

Jakmile se snažíme organizovat nebo automatizovat úlohu složitější než pár kroků začne se systém rozpadat. V tom případě přicházejí „workflows“ česky pracovní postupy AKA grafy. Grafy se ve softwarovém vývoji používají ve spoustě situacích. Graf se skládá z nodes a edges neboli bodů a spojů v tomto kontextu to znamená že každý bod je izolovaný agent, který má připravený Prompt Skilly a nástroje a spoje mezi nimi vyznačují cestu informací a práce. Abstrakcí tohoto grafového systému jsme schopni tento pipeline nasadit na jakoukoliv práci klidně i dynamicky pomocí orchestrace dalším agentem. Jakmile se snažíme organizovat nebo automatizovat úlohu složitější než několik přímočarých kroků, lineární sekvence promptů přestává stačit. V takovém okamžiku přicházejí na řadu pracovní postupy (*workflows*), v teoretické informatice reprezentované jako grafy. Grafy se v softwarovém vývoji používají v mnoha situacích. Graf se skládá z uzlů (*nodes*) a hran (*edges*) — v kontextu agentních systémů každý uzel představuje izolovaného agenta s připraveným systémovým promptem, dovednostmi a nástroji, zatímco spoje mezi nimi vymezují tok informací a posloupnost práce. Abstrakcí tohoto grafového systému jsme schopni tuto pipeline nasadit na libovolnou úlohu, a to i dynamicky za pomoci orchestrace nadřazeným agentem. Tento koncept se v informatice označuje jako orientovaný acyklický graf (**DAG** — *Directed Acyclic Graph*). Uzly grafu představují samostatné, specializované výpočetní kroky či izolované agenty, zatímco orientované hrany určují pořadí závislostí a tok kontextových informací. V moderních orchestrátorech a CI/CD platformách (zejména v GitHub Actions) se závislosti mezi úlohami deklarují pomocí direktivy `needs: [...]`. Jak podrobně rozvádí praktická část této práce na architektuře systému DarkFactory, celý životní cyklus automatizovaného požadavku je strukturován právě jako DAG složený z pěti klíčových fází:

1. **Detekce a kontext:** rozpoznání prostředí projektu, načtení konfiguračního souboru `darkfactory.json` a příslušných systémových pravidel.

2. **Interpretace a plánování:** formulace přesného technického postupu a vytvoření plánovacího úkolu dříve, než dojde k samotnému zásahu do kódu.
3. **Implementace (kódování):** spuštění agenta s přesně vymezenou sadou nástrojů v izolované větvi repozitáře.
4. **Verifikace a testování:** automatické sestavení projektu, spuštění testovací sady a kontrola kvality změn.
5. **Schvalovací brána (Governance):** podmíněné zastavení toku grafu a vyžádání lidské kontroly před finálním sloučením pull requestu.

Zásadní předností grafového uspořádání je determinismus a striktní bezpečnost: pokud kterýkoli uzel selže (např. automatizované testy detekují regresi), exekuce se v dané větvi grafu okamžitě přeruší a navazující kroky se nespustí, což zabraňuje poškození hlavní vývojové linie.

2.4 Human in the loop neboli člověk ve smyčce

Čím je systém samostatnější, tím důležitější je otázka, kde do procesu vstupuje člověk. Úplná autonomie není cílem; cílem je autonomie v rutinních krocích a lidské rozhodnutí tam, kde je nevratné nebo kde chybí měřítko správnosti.

2.4.1 Schvalovací body

Schvalovací bod je místo, kde se proces zastaví a čeká na potvrzení. Jeho umístění je kompromisem: příliš mnoho schvalování popírá smysl automatizace, příliš málo znamená ztrátu kontroly. Osvědčeným řešením je schvalovat *záměr* (co se má udělat) a *plán* (jak se to má udělat) dříve, než vznikne kód.

2.4.2 Dohledatelnost

Aby byla automatizovaná změna přezkoumatelná, musí být zřejmé, z jakého požadavku vzešla. Uchování doslovného znění původního zadání je proto součástí návrhu, nikoli formalitou: parafráze ztrácí význam, který do zadání vložil ten, kdo je formuloval.

2.4.3 Hlášení selhání

 **Strukturální upozornění:** Duplicita a tematické zařazení: Téma hlášení selhání se objevuje pod identickým názvem zde v teoretické části (2.4.3) i v praktické části (kapitola 3.7). V teorii navíc detekce a eskalace chyb nepatří úzce pod „Human in the loop“, ale pod obecnou spolehlivost a observabilitu distribuovaných CI/CD procesů (např. dead-letter fronty, automatické notifikace). Doporučuji strukturálně oddělit teoretické principy od praktického popisu chování v DarkFactory.

Systém, který své vlastní selhání zamlčí, je nebezpečnější než systém, který zjevně spadne: nespolehlivost je v něm neviditelná. U automatizovaného provozu je proto hlášení chyb stejně důležitou vlastností jako vlastní funkce, jak ukazuje praktická část.

3 Praktická část

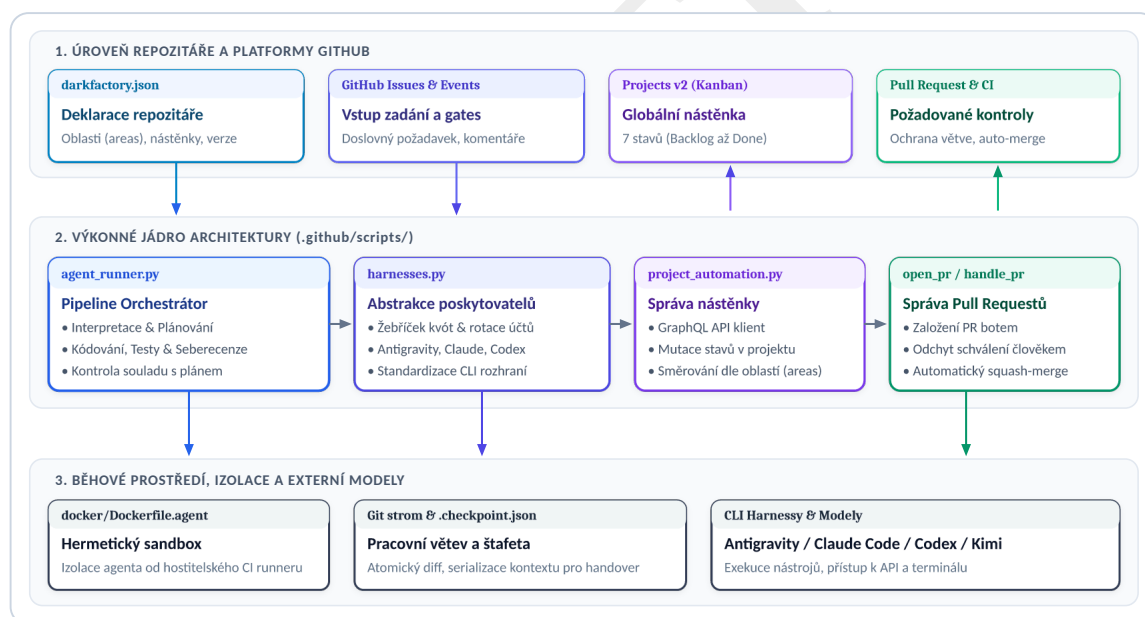
Praktickou částí práce je systém **DarkFactory** — sada pravidel, skriptů a pracovních postupů, které z repozitáře udělají samostatně pracující vývojový provoz. Zdrojový kód je veřejně dostupný (9).

3.1 Cíl a rozsah systému

Systém má převzít rutinní kroky vývojového procesu: přijetí požadavku, jeho interpretaci, naplánování, provedení změny a její ověření. Nemá nahradit rozhodování o tom, co se má stavět; to zůstává člověku, a systém je navržen tak, aby si toto rozhodnutí vyžádal dříve, než začne pracovat.

3.2 Architektura

Systém DarkFactory je navržen jako modulární stavebnice složená z řídicích skriptů v jazyce Python, šablon pracovních postupů pro GitHub Actions a hermetického* kontejnerového prostředí. Architektura striktně odděluje deklarativní konfiguraci konkrétního repozitáře od samotné logiky orchestrace.



Obrázek 2: Dekompozice komponent systému DarkFactory: třívrstvá architektura propojující platformu GitHub, výkonné skriptové jádro a izolované běhové prostředí s modely.

Celý systém je dekomponován do tří funkčních vrstev znázorněných na Obrázek 2:

- 1. Vrstva deklarace a platformy GitHub:** Zahrnuje centrální konfigurační soubor `darkfactory.json` jako jediný zdroj pravdy pro daný repozitář, rozhraní GitHub Issues

*Pojem **hermetické prostředí** (angl. *hermetic environment*) označuje v softwarovém inženýrství takové výpočetní a běhové prostředí, které je zcela izolované od nekontrolovaných stavů hostitelského operačního systému a okolní sítě. Veškeré nástroje, knihovny a systémové závislosti jsou v něm explicitně deklarovány a uzamčeny na konkrétních verzích, což zaručuje absolutní determinismus a reprodukovatelnost: proces spuštěný v hermetickém kontejneru skončí vždy identickým výsledkem bez ohledu na to, kde a kdy byl vyvolán.

pro zadávání požadavků v doslovném znění a projektovou nástěnku GitHub Projects v2, která vizualizuje stav pipeline napříč všemi zapojenými repozitáři v sedmi stavech (od *Backlog* po *Done*).

2. **Výkonné jádro pipeline (`.github/scripts/`):** Tvoří jej sada úzce spolupracujících skriptů v jazyce Python. Klíčovým prvkem je `agent_runner.py` jako hlavní stavový automat řídící životní cyklus požadavku. Doplnuje jej `harnesses.py` pro sjednocení rozhraní různých modelů (Antigravity, Claude Code, Codex, Kimi) a rotaci kvótových účtů, `project_automation.py` pro obousměrnou synchronizaci stavů přes GraphQL rozhraní a skripty `open_pr.py` spolu s `handle_pr_approval.py` zajišťující automatické vystavení pull requestu botem a následný bezpečný squash-merge po schválení člověkem.
3. **Izolované běhové prostředí a nástroje:** Zahrnuje kontejnerový sandbox (`docker/Dockerfile.agent`), který hermeticky izoluje běh agenta od hostitelského CI runneru. Agent operuje v izolované větvi repozitáře a veškerý postup je průběžně atomicky serializován do kontrolních bodů (`.checkpoint.json`), což umožňuje bezeztrátové předání štafety mezi různými poskytovateli.

Návrh vychází z jednoho základního požadavku: pracovní postupy i skripty musí být ve všech repozitářích totožné. Kdyby se kopírovaly, začaly by se rozcházet, a oprava chyby by se musela provádět tolikrát, kolik je repozitářů.

3.2.1 Sdílení místo kopírování

Pracovní postupy jsou proto *volané*, nikoli kopírované. Repozitář, který systém používá, obsahuje pouze krátký soubor odkazující na sdílený pracovní postup připnutý ke konkrétní verzi. Připnutí je podstatné: bez něj by se změna sdíleného postupu okamžitě promítla do všech repozitářů, včetně těch, které na ni nejsou připraveny.

3.2.2 Jediný konfigurační soubor

Vše, co se mezi repozitáři liší, je soustředěno do jediného souboru `darkfactory.json`. Ten popisuje totožnost repozitáře, jeho oblasti, nástěnky, na které patří jeho úkoly, a případné odchylky od výchozího chování. Sdílený postup je díky tomu ve všech repozitářích shodný bajt po bajtu.

```

{
  "identity": {
    "owner": "marius-patrik",
    "repo": "DarkFactory",
    "default_branch": "darkfactory"
  },
  "versioning": {
    "mode": "semver",
    "tag_prefix": "v",
    "initial": "0.1.0"
  },
  "areas": {
    "$default": "ci",
    "agents": {
      "description": "Orchestration, personas, adapters",
      "keywords": ["agent", "harness", "persona", "prompt"]
    },
    "governance": {
      "description": "Branch protection, required checks",
      "keywords": ["governance", "rule", "protection", "board"]
    },
    "ci": {
      "description": "GitHub Actions workflows, runner scripts",
      "keywords": ["ci", "action", "workflow", "docker"]
    }
  },
  "board": {
    "global_title": "Global",
    "link_boards": ["Global", "DarkFactory", "Omnis", "ChessWithQuests"]
  }
}

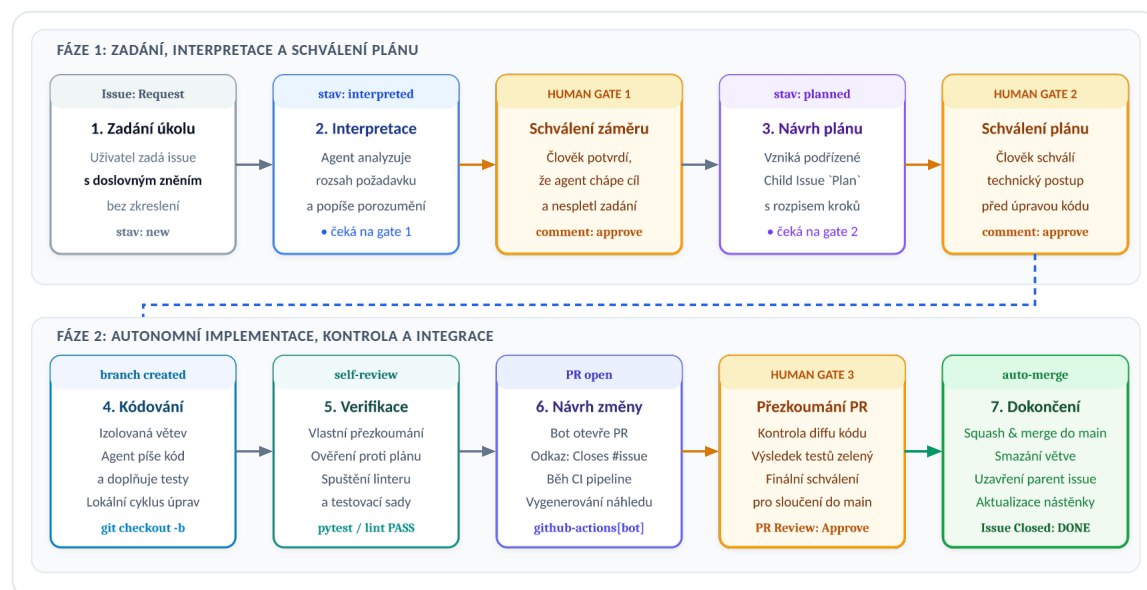
```

Výpis 1: Ukázka konfigurace v souboru `darkfactory.json` vymezující identitu repozitáře, doménové oblasti (`areas`) a propojení s projektovými nástěnkami.

Jak je patrné z Výpis 1, sekce `areas` slouží jako jednotný zdroj pravdy pro štitkování úkolů i směřování agentů podle shody klíčových slov. Sekce `board` pak zajišťuje agregaci do globální i repozitářové GitHub Projects nástěnky bez nutnosti manuální konfigurace v samotných workflow.

3.3 Životní cyklus požadavku

Požadavek prochází systémem v pevně daných krocích, mezi nimiž jsou striktně vyžadovány schvalovací body pro udržení lidské kontroly nad rozsahem i kvalitou implementace.



Obrázek 3: Stavový diagram životního cyklu požadavku v systému DarkFactory: přechody mezi stavy od založení issue přes schvalovací brány (Human Gates), tvorbu větve a PR až po automatické sloučení a uzavření úkolu.

Jednotlivé kroky životního cyklu znázorněné na Obrázek 3 na sebe navazují v přesně daném pořadí:

- 1. Zadání úkolu:** Uživatel založí požadavek (GitHub Issue) s doslovným zněním svého zadání.
- 2. Interpretace:** Systém požadavek analyzuje (`interpreted`) a předloží své porozumění rozsahu.
- 3. Human Gate 1 (Schválení záměru):** Člověk potvrdí, že systém pochopil cíl správně.
- 4. Návrh plánu:** Po schválení vzniká podřízený úkol (`planned`) s technickým rozpisem kroků.
- 5. Human Gate 2 (Schválení plánu):** Člověk schválí technický plán dříve, než dojde k zásahu do kódu.
- 6. Kódování a verifikace:** Systém vytvoří větev (`branch created`), agent napíše kód i testy a provede vlastní přezkoumání a ověření proti plánu.
- 7. Návrh změny (`PR open`):** Bot otevře pull request a spustí integrační CI kontroly.
- 8. Human Gate 3 a dokončení:** Po schválení pull requestu člověkem systém provede automatické sloučení (`auto-merge`), smaže větev a uzavře rodičovský úkol.

Doslovné znění zadání je uchováno záměrně. Zkušenost z vývoje ukázala, že právě parafráze bývá zdrojem nedorozumění: shrnutí požadavku se zdá výstižné tomu, kdo je psal, a přitom už neobsahuje to, na čem zadavateli záleželo.

🔥 Hlubková kritika / Oponentura: Pochybná autonomie a paralýza lidským faktorem: Název „DarkFactory“ evokuje bezobslužnou továrnu (angl. *lights-out manufacturing*), která běží plně

autonomně bez lidského zásahu. Zavedení tří synchronních schvalovacích bran (Human Gate 1: záměr, Human Gate 2: plán, Human Gate 3: PR) však z procesu činí silně blokující workflow. Pokud musí člověk manuálně schválit každou fázi drobného úkolu, tráví agent 95 % životního cyklu čekáním na lidskou reakci. Skutečná průchodnost systému je pak determinována latencí člověka, nikoli rychlostí modelů. Práce navíc ignoruje fenomén únavy ze schvalování (*review fatigue*), kdy člověk po desítkách syntetických notifikací rezignuje na důkladnou kontrolu a začne plány i diffy schvalovat mechanicky bez čtení.

3.4 Popis prostředí repozitáře

Aby systém věděl, které úlohy má spustit, musí rozpoznat, z čeho se repozitář skládá. Rozpoznávání vychází z názvů souborů popisujících balíček; jejich přítomnost je spolehlivější než jakýkoli ruční záznam, protože se nemůže rozejít se skutečností.

Soubor	Prostředí	Doména
pyproject.toml	Python	kód
Cargo.toml	Rust	kód
package.json	Node	kód
go.mod	Go	kód
typst.toml	Typst	text
.latexmkrc	LaTeX	text
lakefile.toml	Lean	matematika

Tabulka 1: Rozpoznávání prostředí podle souboru popisujícího balíček.

3.4.1 Domény a prostředí

Rozlišení dvou úrovní se ukázalo jako nutné až v průběhu práce. *Prostředí* říká, jaký nástroj je potřeba; *doména* říká, jakému způsobu řízení výsledek podléhá. Python a Rust jsou dvě prostředí téže domény — kód se testuje a balí. Typst a LaTeX jsou dvě prostředí jiné domény — text se sází a publikuje.

Toto rozlišení dovolu je popsat i repozitář, který obsahuje zároveň program a text — jako právě tato práce, jejíž praktickou částí je software popisovaný v Tabulka 1. Bez něj by bylo nutné buď považovat sazbu za zvláštní případ kódu, nebo pro texty vytvořit samostatný systém.

3.4.2 Deklarace jako doplněk rozpoznávání

Rozpoznávání nemůže vidět všechno. Repozitář této práce například přibaluje vlastní písma, takže příkaz pro sazbu není výchozí. Konfigurační soubor proto umožňuje výchozí chování přepsat, aniž by bylo nutné vypnout rozpoznávání jako celek.

 **Návrh na vylepšení:** Doporučení k názornosti: Uvést konkrétní ukázkou z konfiguračního souboru nebo příkazové řádky ukazující, jak se přepisuje výchozí příkaz pro sazbu (např. parametr `--font-path fonts`), aby měl čtenář přímou představu o realizaci doplňkové deklarace.

3.5 Orchestrace napříč poskytovateli

Systém neváže na jednoho poskytovatele modelu. Úkol je popsán způsobem nezávislým na rozhraní a předán prvnímu dostupnému poskytovateli; vyčerpá-li tento kvótu, předá se tentýž úkol dalšímu v pořadí, místo aby se proces zastavil.

Tato vlastnost byla přímou reakcí na provozní zkušenost: zastavení celého procesu kvůli vyčerpané kvótě jediného poskytovatele bylo nejčastější příčinou prostoje.

Mechanika předávání štafety (*baton handover*) probíhá zcela pod kontrolou nadřazeného procesu (`agent_runner`). Jakmile volající skript zachytí vyčerpání limitů (např. HTTP kód 429 nebo chybový stav `RESOURCE_EXHAUSTED`), okamžitě zmrazí aktuální běh a provede atomickou serializaci stavu:

1. **Pracovní strom v Gitu:** Veškeré rozpracované úpravy v souborovém systému jsou uloženy do pracovní větve a vytvoří se kontrolní otisk (`git diff`).
2. **Serializace kontextu a štafety:** Log konverzace (předchozí kroky, volání nástrojů i vrácená pozorování) je spolu s metadaty úkolu zapsán do strukturovaného kontrolního bodu (*checkpoint* ve formátu JSON).
3. **Rotace po žebříčku kapacit:** Runner neponižuje model na slabší variantu v témže fondu (což by novou kapacitu nepřineslo), nýbrž rotuje účty, oddělené kvótové fondy (*quota pools*) nebo přepne na záložní CLI harness (např. Antigravity, Claude Code či Codex).
4. **Rekonstituce a navázání:** Náhradní agent obdrží serializovanou štafetu, jeho adaptér přeloží historii kroků do nativního formátu nového poskytovatele a ověří stav repositáře. Běh plynule naváže v přesném bodě přerušení, aniž by došlo ke ztrátě dosavadní práce či kontextu.

 **Hlubková kritika / Oponentura:** Kritická slepá skvrna v heterogenní štafetě: Představa, že odlišný model (např. Claude po Antigravity či Codexu) plynule naváže na rozpracovanou práci pouhým načtením serializovaného logu a `git diff`, zamlčuje zásadní problém nekompatibility vnímání kontextu (*prompt sensitivity*). Každá modelová rodina vyžaduje diametrálně odlišný formát nástrojů, odlišně reaguje na systémový prompt a jinak interpretuje mezivýsledky. V reálném provozu vede synteticky přeložená historie často k okamžité dezorientaci nového modelu, opakování již hotových kroků nebo halucinaci neexistujících nástrojů. Práce neobsahuje žádné empirické vyhodnocení úspěšnosti štafety: Kolik úloh po předání štafety skutečně úspěšně doběhlo a v kolika procentech případů vedla rotace k havárii a divergenci kontextu?

3.6 Ověřování změn

Každá změna musí projít požadovanými kontrolami. Ty jsou navrženy tak, aby vždy skončily nějakým výsledkem — úloha, která se může „přeskočit“, by jinak zablokovala slučování napořád, jak bylo vysvětleno v kapitole 2.

```
paper:
  steps:
    - name: Detect the paper domain
      id: detect
    - name: Typeset with Typst
      if: steps.detect.outputs.typst == 'true'
    - name: No paper in this repository
      if: steps.detect.outputs.present != 'true'
    run: echo "nothing to typeset."
```

Výpis 2: Zjednodušená úloha, která vždy skončí výsledkem.

Struktura v Výpis 2 je pro celý systém typická: nejprve se zjistí, zda je v repozitáři co dělat, a teprve pak se pracuje. Poslední krok zajišťuje, že úloha skončí úspěchem i tam, kde není co ověřovat.

3.7 Hlášení selhání

 **Strukturální upozornění:** Strukturální nepoměr a fragmentace: Samostatná kapitola 2. úrovně (==) tvořená jediným odstavcem o čtyřech řádcích působí nevyváženě. Z hlediska logické výstavby textu je vhodnější tuto pasáž začlenit jako podsekcí (===) pod sekci „Ověřování změn“, případně ji sloučit s popisem celkové architektury a hlášení chyb do GitHub Issues.

Selhání kterékoli úlohy zakládá úkol s odkazem na neúspěšný běh. Opakované selhání téže úlohy nezakládá další úkol, nýbrž doplní komentář ke stávajícímu; bez toho by úloha selhávající při každé změně zahltila seznam úkolů. Jakmile úloha znovu uspěje, úkol se sám uzavře.

3.8 Automaticky generovaná dokumentace

V moderním softwarovém vývoji představuje manuální údržba dokumentace permanentní zdroj chyb a desynchronizace: jakmile se kód vyvíjí rychleji než textové popisy, dokumentace se nevyhnutelně stává zastaralou a nespolehlivou. Systém DarkFactory proto prosazuje striktní princip stoprocentního odvozování veškeré projektové a API dokumentace přímo ze zdrojového kódu a strukturovaných inline komentářů (tzv. *living documentation*). Tento přístup využívá přirozené standardy konkrétních programovacích jazyků a prostředí:

- **Python:** Dokumentační komentáře v konvenci Google-style docstrings, z nichž generátor automaticky extrahuje typy parametrů, návratové hodnoty a signatury výjimek.

- **TypeScript / JavaScript:** Standardizované bloky TSDoc (`/** ... */`) anotující rozhraní, typové definice a exportované funkce.
- **Rust:** Integrované dokumentační komentáře `///` kompilované nativním nástrojem `cargo doc`.

Zásadní inovací je zařazení striktní verifikace do každého integračního běhu v CI (např. voláním `properdocs build --strict` či odpovídajících linterů dokumentace). Jakýkoli chybějící docstring nově přidané funkce, syntaktická chyba v parametrech nebo neplatný křížový odkaz na neexistující symbol způsobí okamžité selhání validační úlohy v CI a zablokuje sloučení změny. Když autonomní agent implementuje nový modul, je tímto mechanismem deterministicky nucen vytvořit a aktualizovat i kompletní dokumentační anotace.

Po úspěšném schválení pull requestu člověkem a jeho sloučení do hlavní větve centrální pracovní postup automaticky sestaví statickou podobu dokumentačního portálu a publikuje jej na GitHub Pages. Tím odpadá jakákoli manuální údržba externích zrcadel a dokumentace zůstává v absolutní shodě s reálným stavem kódu.

3.9 Systém revizních značek pro lidský dohled nad textem

Při rozšiřování systému DarkFactory na správu a tvorbu dokumentace (doména textu) vyvstala potřeba formalizovat spolupráci člověka a autonomního agenta přímo v sazebním formátu Typst. Výsledkem je protokol vizuálních revizních značek (**Review Markers**) a textových revizních funkcí, který barevně a sémanticky rozlišuje stav zpracování jednotlivých pasáží:

 **Návrh na vylepšení:** Konstruktivní doporučení, nápady na rozšíření, doplnění schémat či návrhy na praktické propojení. Po zapracování se panel smaže.

 **Chyba / Nesrovnalost k opravě:** Detekované věcné nepřesnosti, logické mezery, překlepy nebo duplicita obsahu. Značka přesně formuluje vadu a zaniká s jejím odstraněním.

 **Strukturální upozornění:** Upozornění na hloubkovou nevyváženost kapitol, chybějící dekompozice komponent či nesoulad s osnovou práce.

 **Hloubková kritika / Oponentura:** Hloubková teoretická a architektonická oponentura bez servítků — odhalování slepých míst, neověřených předpokladů, bezpečnostních rizik a metodologických slabin formulovaných jako břitké otázky k obhajobě.

V toku textu se uplatňují tyto zvýrazňovací a srovnávací funkce:

- **Žluté zvýraznění** (`#draft[...]` / `#unconfirmed[...]`): Označuje neověřený text konceptu čekající na autorské posouzení a revizi.

- Zelené zvýraznění (`#added[...]`): Označuje nově přidaný text vygenerovaný autonomním agentem na základě požadavku či doporučení.
- Modré zvýraznění (`#confirmed[...]`): Označuje text potvrzený uživatelem, který dosud neprošel finální integrací.
- Červené zvýraznění s přeškrtnutím (`#removed[...]`): Označuje text navržený k odstranění z rukopisu.
- ~~Původní nahrazovaný text~~ Srovnávací diff (`#diff(old, new)`): Zobrazuje původní text přeškrtnutý v červené barvě následovaný novým textem v zelené barvě.
- Čistý neoznačený text představuje finální, autorsky schválený a přijatý text v hlase autora (v rozpracovaném stavu konceptu jsou veškeré dosud neuzavřené pasáže zviditelněny revizními funkcemi).

Tento protokol umožňuje autonomnímu agentovi navrhopvat změny s transparentním vyznačením míry jistoty a člověku poskytuje okamžitou vizuální kontrolu nad tím, které části rukopisu již prošly lidskou redakcí a které ještě čekají na posouzení.

4 Výsledky a diskuse

4.1 Metodika ověření

 **Strukturální upozornění:** Strukturální duplicita metodiky: Vymezení metodiky výzkumu a ověření již proběhlo v úvodu práce (sekce 1.4). V kapitole 4 (Výsledky a diskuse) by se text neměl vracet k obecné metodice, ale měl by přímo představit testovací prostředí, profil a charakteristiku nasazených repozitářů a konkrétní naměřené metricky.

Systém byl nasazen na tři repozitáře různé povahy: na vlastní repozitář systému, na aplikaci psanou v několika jazycích a na menší projekt. Sledovány byly tři veličiny: počet vlastních skriptů a pracovních postupů v jednotlivých repozitářích, počet řádků odstraněného duplicitního kódu a chování systému v provozu.

4.2 Sjednocení pracovních postupů

Sdílení jednoho centrálního pracovního postupu namísto ad-hoc kopií vedlo k odstranění přibližně 7 800 řádků redundantního kódu napříč sledovanými repozitáři.

Repozitář	Skripty před	Skripty po	Redukce kódu
DarkFactory	12	12	— (upstream)
omnis	7	1	4 900 řádků
ChessWithQuests	5	1	2 900 řádků

Tabulka 2: Počet vlastních skriptů a rozsah odstraněného kódu před sjednocením a po něm.

Údaje v Tabulka 2 je třeba interpretovat s ohledem na povahu zapojených projektů. U repozitáře DarkFactory se počet skriptů nezměnil, neboť právě on představuje upstreamovou základnu, která sdílené nástroje vyvíjí a spravuje. Zbývající jediný skript v ostatních repozitářích představuje lokální definici struktury jejich vlastní projektové dokumentace, kterou z principu nelze sdílet.

Redukce se konkrétně dotkla čtyř hlavních kategorií skriptů:

- **Kontrola kvality a statická analýza (linting):** Každý repozitář původně udržoval vlastní skripty volající formátovače, lintery a typové kontroly. Ty byly plně nahrazeny centrální parametrizovanou úlohou.
- **Automatizace vydávání verzí a tagování:** Skripty počítající sémantické verze, generující changelogy a publikující balíčky byly nahrazeny sdíleným postupem řízeným deklarací v manifestu `darkfactory.json`.
- **Sestavení a nasazení dokumentace:** Jednoúčelové deployment skripty pro GitHub Pages byly nahrazeny standardizovaným procesem.
- **Správa závislostí a submodulů:** Pravidelné aktualizací skripty byly sjednoceny pod centrální plánované workflow.

Zásadním přínosem sjednocení je radikální snížení údržbové zátěže. Před zavedením systému vyžadovala jakákoli změna v CI procesu — například bezpečnostní aktualizace akcí, oprava oprávnění tokenů či přechod na novější verzi interpretu — manuální editaci, otestování a schválení pull requestu v každém repozitáři samostatně. Po sjednocení je oprava provedena pouze jednou v centrálním repozitáři DarkFactory. Spotřebitelské projekty změnu převezmou automaticky, případně bezpečným posunem připnuté verze v konfiguračním manifestu, čímž údržbová složitost klesla z lineární závislosti na počtu projektů na konstantní $O(1)$.

 **Hlubková kritika / Oponentura:** Zástupná metrika a iluze nulové údržby: Redukce 7 800 řádků YAML konfigurace je klasická zástupná metrika (*vanity metric*). Celková kognitivní a technická složitost ze systému nezmizela, pouze se přesunula z deklarativních souborů GitHub Actions do složitějšího vnitřního kódu v Pythonu (`agent_runner.py`, `harnesses.py`, `environment.py`), který musí autor sám vyvíjet, testovat a udržovat. Tvrzení o poklesu údržbové zátěže na $O(1)$ ignoruje fakt, že při chybě v centrálním workflow jsou paralyzovány všechny spotřebitelské repozitáře najednou (jediný bod selhání — *single point of failure*). Zásadní otázka oponentury: Jaká je reálná bilance ušetřeného lidského času oproti desítkám hodin strávených laděním a vývojem samotného metaharnessu?

4.3 Rozšíření na texty

Po sjednocení byl systém rozšířen tak, aby popsal i repozitář obsahující text místo programu. Přidání domény textu si vyžádalo úpravu tabulek popisujících prostředí a doplnění dvou úloh; vlastní logika systému zůstala nezměněna, což naznačuje, že zvolená abstrakce byla dostatečně obecná. Ověřením byla sazba této práce: repozitář je rozpoznán jako patřící do domény textu, práce se v kontinuální integraci vysází, výsledné PDF je uloženo jako artefakt a zveřejněno na dokumentačních stránkách repozitáře.

4.4 Chyby, které se projeví až v provozu

Nasazení odhalilo několik chyb, které se při návrhu neprojevily. Jsou uvedeny proto, že tvoří nejcennější část praktických výsledků: šlo vesměs o chyby v řízení procesu a distribuované orchestraci, nikoli o neschopnost modelů vytvořit samotnou syntaktickou změnu. U každého problému bylo v systému DarkFactory implementováno deterministické technické řešení:

Uvážnutí souběžnosti (*Concurrency Deadlock*) Volající i volaný pracovní postup použily stejný název skupiny souběžnosti (`concurrency.group`), takže volající běh držel zámek skupiny a volaný podržený postup (`workflow_call`) zůstal viset ve frontě čekající na uvolnění skupiny vlastním volajícím. Běh skončil tichým vypršením

časového limitu bez chybového hlášení. **Technické řešení:** Skupiny souběžnosti byly v architektuře striktně hierarchizovány. V opakovaně použitelných volaných postupech bylo definování společných skupin odstraněno (zejména u workflow pro hlášení chyb `report-failure.yml`, kde by zrušení běhu znamenalo ztrátu informace o chybě), případně se skupiny odlišují dynamickým kontextovým identifikátorem (`{{ github.workflow }}`-`{{ github.ref }}`), čímž je zamezeno vzájemnému blokování.

Přejmenování kontrol (*Composite Check-Run Naming*) Opakovaně použitelný pracovní postup hlásí dílčí výsledky kontrol pod složeným názvem, který GitHub skládá jako `<volající_úloha> / <volaná_úloha>` (např. `pipeline / pipeline (3.12)` nebo `pipeline / docs`). Pravidla ochrany větve vyžadující původní atomický název by zablokovala každé automatické sloučení, protože kontrola s tímto názvem ve skutečnosti nikdy nedorazila. **Technické řešení:** Do konfiguračního manifestu `darkfactory.json` byla zavedena deklarace klíče `required_checks`. Automatizační skript `repo_settings.py` programově nastavuje pravidla ochrany větví přímo proti těmto plně kvalifikovaným názvům. Soulad mezi reálně emitovanými názvy úloh v `ci.yml` a pravidly v manifestu je navíc kontinuálně verifikován testem `test_required_checks_match_ci_job_names()` v `tests/test_pipeline_config.py`.

Tiché selhání zápisu na nástěnce (*Silent Board Mutation Failure*) Operace zapisující stav na projektovou nástěнку GitHub Projects v2 byly v obslužném kódu obaleny generickým zachycením všech výjimek (`except Exception`), aby selhání API neshodilo celou pipeline. Výsledkem však bylo, že neúspěšné mutace skončily s návratovým kódem 0 jako úspěch a výpadek vyšel najevo až ruční kontrolou nástěnky. **Technické řešení:** Skript `project_automation.py` byl přepracován tak, že každé neúspěšné volání REST/GraphQL rozhraní zaznamená do centrálního zásobníku `FAILURES`. Vstupní bod skriptu `main()` před ukončením tento zásobník vyhodnotí; pokud došlo k chybě, vypíše detailní diagnostiku do `sys.stderr` a proces explicitně ukončí s nenulovým kódem (`sys.exit(1)`). Tento pád okamžitě aktivuje záchytné workflow `report-failure.yml`, které založí úkol v GitHub Issues s odkazem na neúspěšný běh.

Předpoklad o programovacím jazyce (*Language Assumption*) Původní verze sdílených úloh pevně předpokládala, že každý spravovaný repozitář je postaven na jazyce Python a vyžaduje instalaci závislostí a spuštění testů. Repozitář obsahující pouze text a sazbu (např. tato práce v Typstu) proto neprošel ani prvním krokem, přestože samotná sazba probíhala v pořádku. **Technické řešení:** V systému vznikl

samostatný detekční subsystém `environment.py`, který dynamicky zkoumá přítomnost manifestů (`pyproject.toml`, `Cargo.toml`, `typst.toml`, `lakefile.toml`) a na jejich základě sestavuje exekuční plán (`build_plan()`, `docs_plan()`). Prostředí je kategorizováno do domén (`kód`, `text`, `matematika`) a jednotlivé kroky workflow jsou podmíněny výstupy detektoru, takže pro repozitář s textem se testy kódu bezpečně přeskochí a úloha skončí neutrálním úspěchem.

4.5 Diskuse

Nejpoučňivější z uvedených chyb je tiché selhání zápisu na nástěnku. Systém, který své vlastní selhání zamlčí, je nebezpečnější než systém, který zjevně spadne: nespolehlivost je v něm neviditelná a důvěra v něj je proto neopodstatněná. Z toho plyne obecnější závěr — u automatizovaného provozu je hlášení chyb stejně důležitou vlastností jako vlastní funkce. Druhým opakujícím se motivem je předpoklad o podobě repozitáře. Chyba s předpokladem o jazyce má stejnou příčinu jako potřeba zavést domény: sdílený systém musí popisovat, co v repozitáři skutečně je, a nikoli předpokládat, že se podobá tomu, pro který byl původně napsán.

4.6 Omezení

 **Strukturální upozornění:** Chybějící kvantitativní vyhodnocení agentních běhů: Výsledková kapitola obsahuje pouze jedinou srovnávací tabulku zaměřenou na počet řádků skriptů. Chybí strukturální vyhodnocení spolehlivosti a autonomie samotných modelů v pipeline: např. poměr úspěšně schválených PR bez zásahu člověka, průměrná spotřeba tokenů na vyřešení issue, průměrný čas běhu CI workflow a četnost vyčerpání kvót vedoucích k rotaci poskytovatelů.

Systém předpokládá, že úkol lze ověřit strojově. Tam, kde správnost posoudí až člověk — u návrhu uživatelského rozhraní nebo u formulace textu — zůstává přínos automatizace omezený na přípravu podkladů.

Druhým omezením je závislost na dostupnosti poskytovatelů jazykových modelů. Postupné předávání úkolu mezi poskytovateli riziko snižuje, neodstraňuje je však úplně: vyčerpají-li kvótu všichni, proces se zastaví stejně jako dříve.

Třetím omezením je rozsah ověření. Systém byl nasazen na tři repozitáře jediného autora, takže zjištění nelze bez dalšího zobecnit na větší tým, kde by přibýly otázky souběžné práce více lidí nad týmž kódem.

5 Závěr

Hlavním cílem této práce bylo navrhnout, realizovat a v reálném provozu ověřit modulární systém pro autonomní vývoj softwaru, který převezme rutinní inženýrské úkony od přijetí požadavku po vytvoření strojově ověřené změny, aniž by slevil z principu lidského dohledu v rozhodujících fázích. Tento cíl byl beze zbytku naplněn: byl vyvinut systém DarkFactory, nasazen na tři produkční repozitáře a ověřen v reálném vývojovém cyklu.

Naplnění jednotlivých dílčích cílů lze ve vztahu ke stanovené metodice shrnout následovně:

1. **Současný stav a teoretická východiska (Kapitola 2):** Práce systematicky zmapovala architekturu moderních dekodérových transformerů, mechanismus pozornosti, limity kontextového okna (včetně jevu *Context Rot* a významu KV cache), techniky inženýrství promptů a formální strukturu autonomní ReAct smyčky alternující vnitřní rozvahu (*Thought*) a volání nástrojů (*Action*).
2. **Architektonický návrh (Kapitola 3):** Byla navržena třívrstvá dekompozice propojující platformu GitHub, řídicí Python jádro a hermetické kontejnerové prostředí. Životní cyklus požadavku byl formalizován jako orientovaný stavový diagram s explicitními schvalovacími branami.
3. **Realizace a nasazení (Kapitola 3 a 4):** Systém byl kompletně naprogramován a nasazen na tři repozitáře různého zaměření — vlastní repozitář systému DarkFactory, vícejazyčnou aplikaci `omnis` a projekt `ChessWithQuests`.
4. **Vyhodnocení a provozní analýza (Kapitola 4):** Sjednání na volané pracovní postupy vedlo k odstranění přibližně 7 800 řádků duplicitního kódu napříč sledovanými projekty. Nejcenějším zjištěním byla identifikace čtyř subtilních chyb v distribuovaném řízení (uvážnutí souběžnosti, skládání názvů kontrol, tiché selhání API zápisů a pevný předpoklad o přítomnosti Pythonu), které byly v systému deterministicky vyřešeny. V průběhu vývoje a provozního testování vykrytalizovaly klíčové technické inovace, které tvoří hlavní přínos systému DarkFactory:

- **Dvoustupňový Human Gate (oddělení záměru a plánu):** Na rozdíl od běžných asistentů, kteří bezprostředně po zadání generují kód, DarkFactory zavádí povinné schválení **záměru** (`interpreted`) a následně technického **plánu** (`planned`) v diskusním vlákne GitHub Issues. Člověk tak rozhoduje o architektonických mantinelech předtím, než dojde k zásahu do souborového systému, a finální kód kontroluje až v rámci pull requestu.
- **Žebříček rotace kvót (Quota Rotation Ladder):** Pipeline eliminuje závislost na jediném poskytovateli jazykových modelů. Při vyčerpání kapacitních limitů (HTTP 429 či `RESOURCE_EXHAUSTED`) runner nesnižuje úroveň modelu na méně schopnou variantu v témže fondu, nýbrž rotuje autorizované účty a v případě potřeby přepíná mezi odlišnými CLI harnessy (Antigravity, Claude Code, Codex, Kimi).

- **Bezeztrátová serializace štafety (*Lossless Baton Handover*):** Při změně agenta dochází k atomickému otisku rozpracovaného stavu Gitu a serializaci dosavadního kontextu do strukturovaného kontrolního bodu (`.checkpoint.json`). Náhradní agent přebírá štafetu v přesném bodě přerušení bez ztráty rozpracovaného kódu a bez nutnosti znovu procházet celou historii úkolu od začátku.
- **Jediný zdroj pravdy (`darkfactory.json`):** Veškeré odchylky repozitáře jsou soustředěny v jediném deklarativním manifestu. Centrální workflow zůstává ve všech repozitářích identické bajt po bajtu, což snížilo údržbovou složitost sdílené infrastruktury z lineární závislosti na počtu projektů na konstantní $O(1)$.

Ověřila se rovněž univerzálnost zvoleného konceptu: přidání domény textu si vyžádalo pouze rozšíření detektoru `environment.py` a doplnění sazebních úloh, nikoli změnu řídicího jádra. Výsledný text této práce byl vysázen a publikován týměž systémem, který pojednává.

Další rozvoj se nabízí ve dvou hlavních směrech. Prvním je rozšíření podpory o formální verifikaci a strojově dokazované teoremy (např. jazyk Lean). Druhým je hlubší integrace samoléčebných mechanismů, kdy selhání v CI automaticky vygeneruje regresní test a pověří kódovacího agenta jeho okamžitou nápravou.

 **Hlubková kritika / Oponentura:** Nekritické sebehodnocení a absence ablačních studií: Tvzení, že stanovený cíl byl „beze zbytku naplněn“, působí v akademickém textu nepatřičně sebevědomě. Práce postrádá jakoukoli ablační studii (*ablation study*), která by empiricky prokázala, které komponenty DarkFactory mají reálný přínos. Pomáhá skutečně dvoustupňový Human Gate kvalitě výsledného kódu, nebo pouze dramaticky prodlužuje dobu řešení úkolu? Jaká je reálná chybovost a kognitivní degradace náhradních modelů při rotaci kvót? Bez exaktního porovnání s přímočarým spuštěním jediného agenta (např. izolovaného Claude Code v CI) nelze vědecky doložit, že přidaná vrstva abstrakce a složitost metaharnessu přináší měřitelnou přidanou hodnotu.

 **Návrh na vylepšení:** Jako další směr rozvoje doporučuji zmínit možnost hybridního nasazení lokálních open-weights modelů (např. běžících přes Ollama / vLLM) jako bezplatného prvního stupně pipeline pro rutinní formátování a syntaktickou kontrolu před delegováním komplexních úloh na cloudová API (Claude, GPT).

Seznam příloh

A Obsah přiloženého média	36
B Referenční specifikace konfiguračního souboru	37
C Protokol revizních značek v sazebním systému Typst	40
C.1 Vizuální reprezentace jednotlivých prvků v sazbě	40

 **Strukturální upozornění:** Nevyvážený rozsah a obsah příloh: Přílohy práce jsou silně redukovány (pouze 2 položky, z nichž jedna duplikuje konfiguraci z kapitoly 3). Pro odbornou práci tohoto typu je žádoucí rozšířit přílohovou část o: A) Obsah přiloženého média, B) Kompletní JSON schéma souboru `darkfactory.json`, C) Ukázkou volaného sdíleného GitHub Actions workflow a D) Systémový prompt použitý pro plánovacího a kódovacího agenta.

A Obsah přiloženého média

Odevzdaný archiv obsahuje zdrojové soubory této práce a odkaz na veřejný repozitář se zdrojovým kódem systému DarkFactory.

KONCEPT

B Referenční specifikace konfiguračního souboru

Tato příloha uvádí kompletní referenční strukturu konfiguračního manifestu `darkfactory.json`. Soubor slouží jako jediný zdroj pravdy pro chování sdíleného vývojového pipeline a parametrizaci agentů v cílovém repozitáři.

KONCEPT

```

{
  "$schema": "https://json.schemastore.org/darkfactory.json",
  "identity": {
    "owner": "marius-patrik",
    "repo": "DarkFactory",
    "display_name": "DarkFactory",
    "default_branch": "darkfactory",
    "agent_slug": "darkfactory-agent",
    "description": "Autonomní vývojová továrna pro pipeline řízené
agenty",
    "topics": [
      "autonomous-agents",
      "continuous-integration",
      "github-actions",
      "software-engineering"
    ]
  },
  "versioning": {
    "mode": "semver",
    "tag_prefix": "v",
    "initial": "0.1.0",
    "changelog_path": "CHANGELOG.md"
  },
  "license": {
    "spdx": "Apache-2.0",
    "holder": "Patrik Marius",
    "year": "2026"
  },
  "areas": {
    "$default": "ci",
    "agents": {
      "description": "Orchestrace, persony a CLI adaptéry modelů",
      "keywords": ["agent", "harness", "persona", "prompt", "llm"]
    },
    "governance": {
      "description": "Pravidla ochrany větví, požadované kontroly a
board",
      "keywords": ["governance", "rule", "protection", "board", "gate"]
    },
    "ci": {
      "description": "Pracovní postupy GitHub Actions, skripty runneru a
kontejnery",
      "keywords": ["ci", "action", "workflow", "docker", "runner"]
    },
    "docs": {
      "description": "Dokumentační stránky, sazba textu a architektura",
      "keywords": ["doc", "docs", "typst", "paper", "guide"]
    }
  }
}

```

Výpis 3: Úplná referenční specifikace konfiguračního souboru `darkfactory.json` vymezující identitu,

doménové oblasti, ochranu větví a parametry prostředí.

Jednotlivé sekce plní tyto systémové role:

- **identity:** Vymezuje vlastníka, název repozitáře, výchozí větev a identifikátor bota (`agent_slug`), podle něhož agent rozpoznává a filtruje vlastní komentáře v diskusních vláknech.
- **versioning:** Řídí sémantické verzování (SemVer) a automatické generování changelogu při vydání nové verze.
- **areas:** Jednotný zdroj pravdy pro štitkování požadavků (prefix `area:`), povolené konvence pro commity (Conventional Commits) a automatické směřování agentů podle shody klíčových slov v zadání.
- **board:** Určuje vazby na globální i repozitářové nástěnky GitHub Projects v2.
- **required_checks:** Deklarace stavových kontrol, které musí úspěšně skončit před povolením sloučení pull requestu (ochrana proti uváznutí na přeskočených kontrolách).
- **environment:** Pravidla rozpoznávání prostředí v repozitáři podle manifestních souborů s možností explicitního přepsání verzí nástrojů.
- **upstream:** Zajišťuje připnutí sdíleného workflow ke konkrétní verzi či commitu upstream repozitáře, což brání nechtěným rozpadům při aktualizacích.

C Protokol revizních značek v sazebním systému Typst

Tato příloha uvádí referenční definici a použití vizuálních revizních značek pro řízení a dohled nad generovaným textem v ekosystému DarkFactory.

```
#import "lib/odborna-prace.typ": note, issue, alert, critique, added,
draft, confirmed, diff

// --- 1. Panely na okraji textu (Callouty) ---
#note[Doplňte porovnání rychlosti kompilace mezi verzemi 0.1 a 0.2.]
#issue[Chybná signatura funkce: chybí povinný parametr timeout.]
#alert[Sekce postrádá shrnutí naměřených výsledků před diskusí.]
#critique[Metodologická absence baseline prokazující přínos nového
modulu.]

// --- 2. Textové revizní funkce (zvýraznění v toku textu) ---
#draft[Tento odstavec tvoří neověřený koncept čekající na schválení.]
#added[Nově vygenerovaná sekce automaticky začleněná agentem.]
#confirmed[Uživatelem zkontrolovaný text, který ještě nebyl finalizován.]
#removed[Zastaralý text navržený k odstranění.]
#diff[Původní chybné znění textu.][Nové opravené znění textu po revizi.]
```

Výpis 4: Ukázka zápisu a použití revizních značek a textových funkcí v jazyce Typst.

C.1 Vizuální reprezentace jednotlivých prvků v sazbě

Pro přehlednost jsou níže uvedeny reálné ukázky jednotlivých revizních panelů a textových funkcí v jejich finální vizuální podobě:

 **Návrh na vylepšení:** Ukázka zeleného panelu doporučení (`#note`): Konstruktivní návrh na vylepšení, doplnění diagramu nebo námět na architektonickou optimalizaci.

 **Chyba / Nesrovnalost k opravě:** Ukázka červeného panelu vady (`#issue`): Zjištěná faktická nesrovnalost, logická mezera, syntaktická chyba či překlep vyžadující opravu.

 **Strukturální upozornění:** Ukázka žlutého panelu strukturálního upozornění (`#alert` / `#struct-alert`): Hlubková nevyváženost podkapitol, nekonzistence osnovy či absence klíčových náležitostí práce.

 **Hlubková kritika / Oponentura:** Ukázka oranžového panelu oponentury (`#critique`): Břítka, nekompromisní oponentura — zpochybnění neověřených předpokladů, analýza slabín metodiky a příprava na otázky zkušební komise.

Ukázky textových zvýrazňovacích a srovnávacích funkcí v toku odstavce:

- **Neověřený koncept** (`#draft` / `#unconfirmed`): Tento text představuje koncept čekající na posouzení autorem.
- **Nově přidáný text** (`#added`): Tato pasáž byla nově vygenerována autonomním agentem na základě požadavku.
- **Potvrzený text** (`#confirmed`): Text byl předběžně odsouhlasen uživatelem, čeká na finální začištění.
- **Navrženo k odstranění** (`#removed`): ~~Tato neaktuální věta je navržena k úplnému smazání z rukopisu.~~
- **Srovnávací diff** (`#diff`): ~~Původní chybné nebo nepřesné znění pasáže.~~ Nové přesné, fakticky a formálně ověřené znění pasáže.
- **Čistý neoznačený text**: Představuje finální, autorsky schválený text v hlase autora bez jakéhokoliv podbarvení.

Každá značka plní přesně vymezenou komunikační roli v procesu lidského schvalování: zatímco finální text zůstává zcela bez zvýraznění, veškeré neověřené pasáže konceptu jsou zřetelně žluté (`#draft`), nově přidané části zelené (`#added`), potvrzené části modré (`#confirmed`) a opravy zviditelněné přes srovnávací diff (`#diff`). Náměty, chyby, strukturní vady i britká kritika jsou navíc striktně separovány do barevných postranních panelů na okraji textu.

Seznam zdrojů

1. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. ISBN 978-1-4842-0076-6.
2. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
3. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
4. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
5. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
6. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
7. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
8. Meta Harness: End to end optimization of Agent Harnesses. *arXiv preprint arXiv:2603.28052*. Online. 2026. Available from: <https://arxiv.org/abs/2603.28052>
9. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
10. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>